Linear Linked Structure

1. What Are Linear Structures?

2. The Unordered Linked List

3. The Ordered Linked List

4. Other types of Linked list

4.2 Lists

Throughout the discussion of basic data structures, we implemented a dynamic array (`ArrayList`) on top of raw `C++` arrays. However, other types of data structure exist — this chapter builds collections out of **nodes and pointers** instead of contiguous memory.

Throughout the discussion of basic data structures, we implemented a dynamic array (`ArrayList`) on top of raw `C++` arrays. However, other types of data structure exist — this chapter builds collections out of **nodes and pointers** instead of contiguous memory.

A list is a collection of items where each item holds a relative position with respect to the others. More specifically, we will refer to this type of list as an unordered list. We can consider the list as having a first item, a second item, a third item, and so on. For simplicity we will assume that lists cannot contain duplicate items.

The Unordered Linked List

The structure of an unordered list is a collection of items where each item holds a relative position with respect to the others. Some possible unordered list operations are given below:

The structure of an unordered list is a collection of items where each item holds a relative position with respect to the others. Some possible unordered list operations are given below:

- `List()` creates a new list that is empty. It needs no parameters and returns an empty list.
- `is_empty()` tests to see whether the list is empty. It needs no parameters and returns a Boolean value.
- `add(item)` adds a new item to the head of list. It needs the item and returns nothing. Assume the item is not already in the list.

The structure of an unordered list is a collection of items where each item holds a relative position with respect to the others. Some possible unordered list operations are given below:

- `List()` creates a new list that is empty. It needs no parameters and returns an empty list.
- `is_empty()` tests to see whether the list is empty. It needs no parameters and returns a Boolean value.
- `add(item)` adds a new item to the head of list. It needs the item and returns nothing. Assume the item is not already in the list.

- `size()` returns the number of items in the list. It needs no parameters and returns an integer.
- `search(item)` searches for the item in the list. It needs the item and returns a Boolean value.
- `remove(item)` removes the item from the list. It needs the item and modifies the list. Raise an error if the item is not present in the list.

- `append(item)` adds a new item to the end of the list making it the last item in the collection. It needs the item and returns nothing. Assume the item is not already in the list.
- `index(item)` returns the position of item in the list. It needs the item and returns the index. Assume the item is in the list.

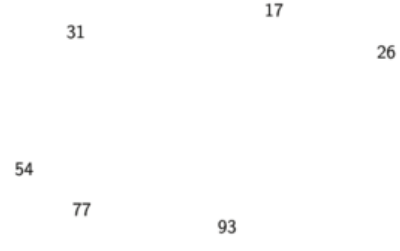
- `append(item)` adds a new item to the end of the list making it the last item in the collection. It needs the item and returns nothing. Assume the item is not already in the list.
- `index(item)` returns the position of item in the list. It needs the item and returns the index. Assume the item is in the list.

- `insert(pos, item)` adds a new item to the list at position pos. It needs the item and returns nothing. Assume the item is not already in the list and there are enough existing items to have position pos.
- `pop()` removes and returns the last item in the list. It needs nothing and returns an item. Assume the list has at least one item.
- `pop(pos)` removes and returns the item at position pos. It needs the position and returns the item. Assume the item is in the list.

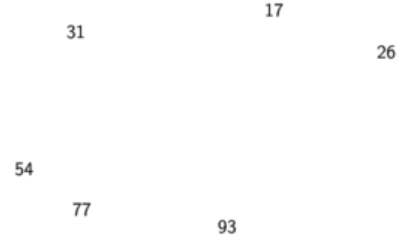
4.3 Implementing an Unordered Linked List: Linked Lists

In order to implement an unordered list, we will construct what is commonly known as a linked list. Recall that we need to be sure that we can maintain the relative positioning of the items. However, there is **no requirement that we maintain that positioning in contiguous memory.**

In order to implement an unordered list, we will construct what is commonly known as a linked list. Recall that we need to be sure that we can maintain the relative positioning of the items. However, there is **no requirement that we maintain that positioning in contiguous memory.**



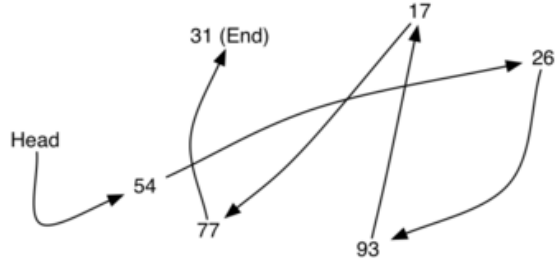
In order to implement an unordered list, we will construct what is commonly known as a linked list. Recall that we need to be sure that we can maintain the relative positioning of the items. However, there is **no requirement that we maintain that positioning in contiguous memory.**



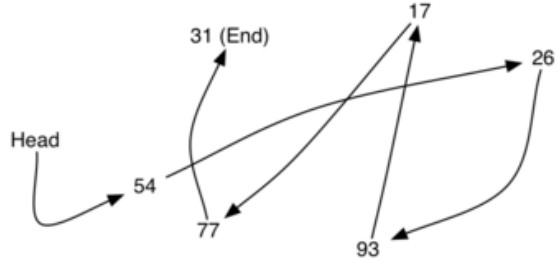
For example, consider the collection of items. It appears that these values have been placed randomly.

If we can maintain some explicit information in each item, namely the **location of the next item**, then the relative position of each item can be expressed by simply following the link from one item to the next.

If we can maintain some explicit information in each item, namely the **location of the next item**, then the relative position of each item can be expressed by simply following the link from one item to the next.



If we can maintain some explicit information in each item, namely the **location of the next item**, then the relative position of each item can be expressed by simply following the link from one item to the next.



It is important to note that the location of the first item of the list must be explicitly specified. Once we know where the first item is, the first item can tell us where the second is, and so on. The external reference is often referred to as the head of the list. Similarly, the last item needs to know that there is no next item.

4.3.1 The `Node` Class

The basic building block for the linked list implementation is the node. Each node object must hold at least two pieces of information. First, the node must contain the list item itself. We will call this the data field of the node. In addition, each node must hold a reference to the next node.

The basic building block for the linked list implementation is the node. Each node object must hold at least two pieces of information. First, the node must contain the list item itself. We will call this the data field of the node. In addition, each node must hold a reference to the next node.

```
template <typename T>
class Node {
    private:
        T data;           // data of generic type
        Node<T> *next;    // pointer to the next node

    public:
        Node(T initdata) {
            data = initdata;
            next = NULL;
        }
        T getData() const {
            return data;
        }
        Node<T> *getNext() const {
            return next;
        }
        void setData(T newData) {
            data = newData;
        }
        void setNext(Node<T> *newnext) {
            next = newnext;
        }
};
```

Note that the fields `data` and `next` of the `Node` class are `private`, and access goes through getter and setter methods (`getData()`, `setNext()`, ...) — the same encapsulation idea Python expressed with properties.

Note that the fields `data` and `next` of the `Node` class are `private`, and access goes through getter and setter methods (`getData()`, `setNext()`, ...) — the same encapsulation idea Python expressed with properties.

We create `Node` objects in the usual way.

Note that the fields `data` and `next` of the `Node` class are `private`, and access goes through getter and setter methods (`getData()`, `setNext()`, ...) — the same encapsulation idea Python expressed with properties.

We create `Node` objects in the usual way.

In [1]:
```cpp
#include <iostream>
#include "dscpp/linked_list.hpp"   // Node / UnorderedList / OrderedList
using namespace std;

int main() {
    Node<int> *temp = new Node<int>(93);
    cout << temp->getData() << endl;
    cout << temp->getNext() << endl;   // NULL prints as 0
    delete temp;
    return 0;
}
```

```
93

0
```

Note that the fields `data` and `next` of the `Node` class are `private`, and access goes through getter and setter methods (`getData()`, `setNext()`, ...) — the same encapsulation idea Python expressed with properties.

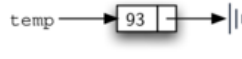
We create `Node` objects in the usual way.

In [1]:
```
#include <iostream>
#include "dscpp/linked_list.hpp"   // Node / UnorderedList / OrderedList
using namespace std;

int main() {
    Node<int> *temp = new Node<int>(93);
    cout << temp->getData() << endl;
    cout << temp->getNext() << endl;   // NULL prints as 0
    delete temp;
    return 0;
}
```

```
93

0
```



The special `C++` pointer value `NULL` will play an important role in the `Node` class and later in the linked list itself. A pointer equal to `NULL` denotes the fact that there is no next node.

Note in the constructor that a node is initially created with `next` set to `NULL`. It is always a good idea to explicitly assign `NULL` to your initial pointer values — an uninitialized pointer contains garbage!

4.3.2 The UnorderedList Class

The unordered list will be built from a collection of nodes, each linked to the next by explicit references. With this in mind, the `UnorderedList` class must maintain a reference to the first node. Code below shows the constructor. Note that each list object will maintain a single reference to the head of the list.

The unordered list will be built from a collection of nodes, each linked to the next by explicit references. With this in mind, the `UnorderedList` class must maintain a reference to the first node. Code below shows the constructor. Note that each list object will maintain a single reference to the head of the list.

```
template <typename T>
class UnorderedList {
    private:
        Node<T> *head;

    public:
        UnorderedList() {
            head = NULL;
        }
};
```

The unordered list will be built from a collection of nodes, each linked to the next by explicit references. With this in mind, the `UnorderedList` class must maintain a reference to the first node. Code below shows the constructor. Note that each list object will maintain a single reference to the head of the list.

```
template <typename T>
class UnorderedList {
    private:
        Node<T> *head;

    public:
        UnorderedList() {
            head = NULL;
        }
};
```

An empty list is created with `UnorderedList<int> myList;` — the constructor sets `head` to `NULL`, so the list starts with no nodes.

The unordered list will be built from a collection of nodes, each linked to the next by explicit references. With this in mind, the `UnorderedList` class must maintain a reference to the first node. Code below shows the constructor. Note that each list object will maintain a single reference to the head of the list.

```
template <typename T>
class UnorderedList {
    private:
        Node<T> *head;

    public:
        UnorderedList() {
            head = NULL;
        }
};
```

An empty list is created with `UnorderedList<int> myList;` — the constructor sets `head` to `NULL`, so the list starts with no nodes.

The assignment statement creates the linked list representation

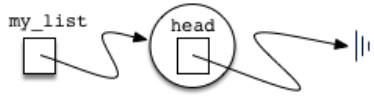
The unordered list will be built from a collection of nodes, each linked to the next by explicit references. With this in mind, the `UnorderedList` class must maintain a reference to the first node. Code below shows the constructor. Note that each list object will maintain a single reference to the head of the list.

```
template <typename T>
class UnorderedList {
    private:
        Node<T> *head;

    public:
        UnorderedList() {
            head = NULL;
        }
};
```

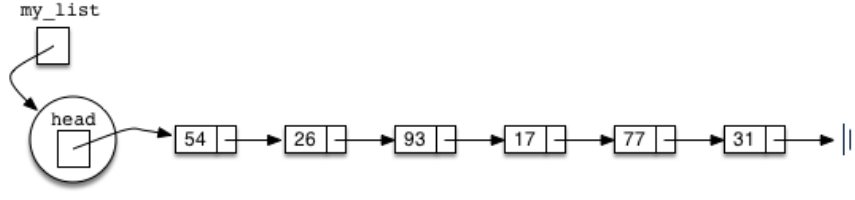
An empty list is created with `UnorderedList<int> myList;` — the constructor sets `head` to `NULL`, so the list starts with no nodes.

The assignment statement creates the linked list representation

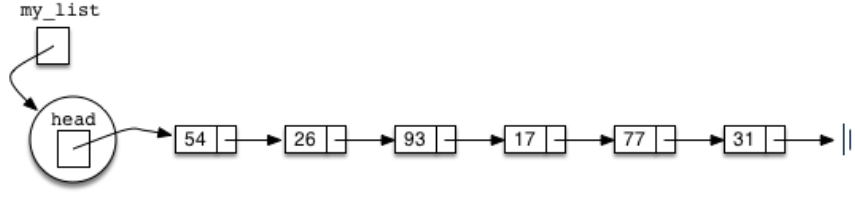


As we discussed in the `Node` class, the special reference `NULL` will again be used to state that the head of the list does not refer to anything. Eventually, the example list given earlier will be represented by a linked list as shown below:

As we discussed in the `Node` class, the special reference `NULL` will again be used to state that the head of the list does not refer to anything. Eventually, the example list given earlier will be represented by a linked list as shown below:



As we discussed in the `Node` class, the special reference `NULL` will again be used to state that the head of the list does not refer to anything. Eventually, the example list given earlier will be represented by a linked list as shown below:



The head of the list refers to the first node which contains the first item of the list. In turn, that node holds a reference to the next node (the next item), and so on.

It is very important to note that the `UnorderedList` class itself does not contain any node objects. Instead it contains a single reference to only the first node in the linked structure.

The `is_empty()` method, shown below, simply checks to see if the head of the list is a pointer to `NULL`. The result of the boolean expression `head == NULL` will only be true if there are no nodes in the linked list.

The `is_empty()` method, shown below, simply checks to see if the head of the list is a pointer to `NULL`. The result of the boolean expression `head == NULL` will only be true if there are no nodes in the linked list.

```
bool isEmpty() const {
    return head == NULL;
}
```

So how do we get items into our list? We need to implement the `add()` method. However, before we can do that, we need to address the important question of where in the linked list to place the new item.

So how do we get items into our list? We need to implement the `add()` method. However, before we can do that, we need to address the important question of where in the linked list to place the new item.

Since this list is unordered, the specific location of the new item with respect to the other items already in the list is not important. The new item can go anywhere. With that in mind, **it makes sense to place the new item in the easiest location possible.**

So how do we get items into our list? We need to implement the `add()` method. However, before we can do that, we need to address the important question of where in the linked list to place the new item.

Since this list is unordered, the specific location of the new item with respect to the other items already in the list is not important. The new item can go anywhere. With that in mind, **it makes sense to place the new item in the easiest location possible.**

Recall that the linked list structure provides us with only one entry point, the head of the list. All of the other nodes can only be reached by accessing the first node and then following next links. This means that the easiest place to add the new node is right at the head, or beginning, of the list

```
my_list.add(31)
my_list.add(77)
my_list.add(17)
my_list.add(93)
my_list.add(26)
my_list.add(54)
```

```
my_list.add(31)
my_list.add(77)
my_list.add(17)
my_list.add(93)
my_list.add(26)
my_list.add(54)
```

Note that since 31 is the first item added to the list, it will eventually be the last node on the linked list as every other item is added ahead of it. Also, since 54 is the last item added, it will become the data value in the first node of the linked list.

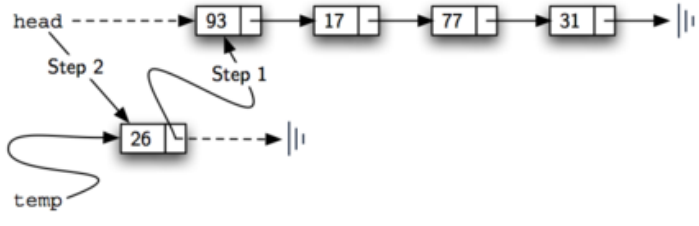
Therefore, we can create a new node and places the item as its data. Now we must complete the process by linking the new node into the existing structure.

Therefore, we can create a new node and places the item as its data. Now we must complete the process by linking the new node into the existing structure.

```
void add(T item) {
    Node<T> *temp = new Node<T>(item);
    temp->setNext(head);
    head = temp;
}
```

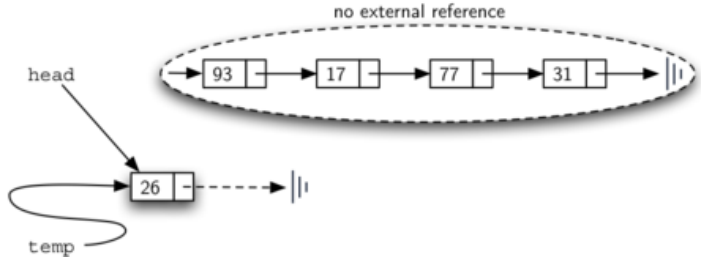
Therefore, we can create a new node and places the item as its data. Now we must complete the process by linking the new node into the existing structure.

```
void add(T item) {
    Node<T> *temp = new Node<T>(item);
    temp->setNext(head);
    head = temp;
}
```

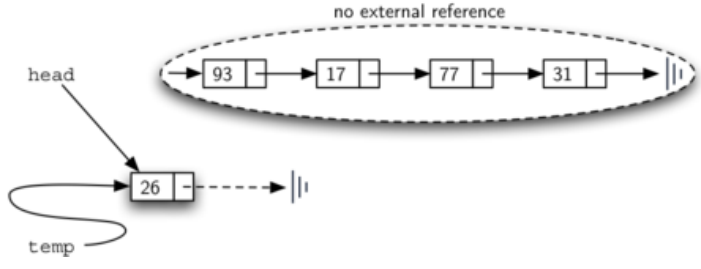


The order of the two steps described above is very important. **What happens if the order of line 3 and line 4 is reversed?**

The order of the two steps described above is very important. **What happens if the order of line 3 and line 4 is reversed?**



The order of the two steps described above is very important. **What happens if the order of line 3 and line 4 is reversed?**



Since the head was the only external reference to the list nodes, all of the original nodes are lost and can no longer be accessed.

The next methods that we will implement – `size()`, `search()`, and `remove()` – are all based on a technique known as linked list traversal. Traversal refers to the process of systematically visiting each node.

The next methods that we will implement – `size()`, `search()`, and `remove()` – are all based on a technique known as linked list traversal. Traversal refers to the process of systematically visiting each node.

To do this we use an external reference that starts at the first node in the list. As we visit each node, we move the reference to the next node by "traversing" the next reference.

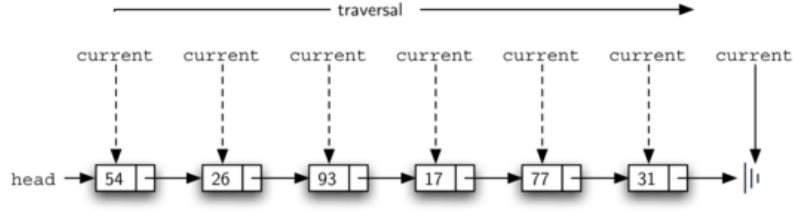
To implement the `size()` method, we need to traverse the linked list and keep a count of the number of nodes that occurred.

To implement the `size()` method, we need to traverse the linked list and keep a count of the number of nodes that occurred.

```
int size() const {
    Node<T> *current = head;
    int count = 0;
    while (current != NULL) {
        count++;
        current = current->getNext();
    }
    return count;
}
```

To implement the `size()` method, we need to traverse the linked list and keep a count of the number of nodes that occurred.

```
int size() const {
    Node<T> *current = head;
    int count = 0;
    while (current != NULL) {
        count++;
        current = current->getNext();
    }
    return count;
}
```



Searching for a value in a linked list implementation of an unordered list also uses the traversal technique. As we visit each node in the linked list we will ask whether the data stored there matches the item we are looking for.

Searching for a value in a linked list implementation of an unordered list also uses the traversal technique. As we visit each node in the linked list we will ask whether the data stored there matches the item we are looking for.

```
bool search(T item) const {
    Node<T> *current = head;
    while (current != NULL) {
        if (current->getData() == item) {
            return true;
        }
        current = current->getNext();
    }
    return false;
}
```

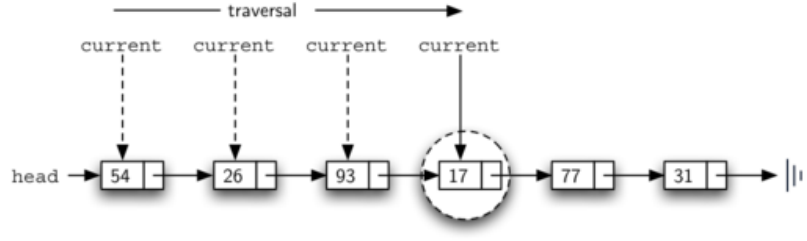
Searching for a value in a linked list implementation of an unordered list also uses the traversal technique. As we visit each node in the linked list we will ask whether the data stored there matches the item we are looking for.

```
bool search(T item) const {
    Node<T> *current = head;
    while (current != NULL) {
        if (current->getData() == item) {
            return true;
        }
        current = current->getNext();
    }
    return false;
}
```

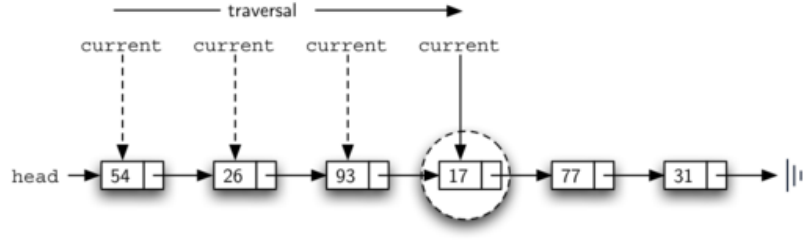
If we do get to the end of the list, that means that the item we are looking for must not be present. Also, if we do find the item, there is no need to continue.

```
my_list.search(17)
```

```
my_list.search(17)
```



```
my_list.search(17)
```



Since 17 is in the list, the traversal process needs to move only to the node containing 17. At that point, the condition in line 4 becomes `True` and we return the result of the search.

The `remove()` method requires two logical steps. First, we need to traverse the list looking for the item we want to remove. Once we find the item, we must remove it. If the item is not in the list, our method should raise a `ValueError`.

The `remove()` method requires two logical steps. First, we need to traverse the list looking for the item we want to remove. Once we find the item, we must remove it. If the item is not in the list, our method should raise a `ValueError`.

The first step is very similar to search. Starting with an external reference set to the head of the list, we traverse the links until we discover the item we are looking for.

The `remove()` method requires two logical steps. First, we need to traverse the list looking for the item we want to remove. Once we find the item, we must remove it. If the item is not in the list, our method should raise a `ValueError`.

The first step is very similar to search. Starting with an external reference set to the head of the list, we traverse the links until we discover the item we are looking for.

When the item is found and we break out of the loop, `current` will be a reference to the node containing the item to be removed. But how do we remove it?

In order to remove the node containing the item, we need to modify the link in the previous node so that it refers to the node that comes after current. Unfortunately, there is no way to go backward in the linked list. Since `current` refers to the node ahead of the node where we would like to make the change, it is too late to make the necessary modification!

In order to remove the node containing the item, we need to modify the link in the previous node so that it refers to the node that comes after current. Unfortunately, there is no way to go backward in the linked list. Since `current` refers to the node ahead of the node where we would like to make the change, it is too late to make the necessary modification!

The solution to this dilemma is to use two external references as we traverse down the linked list. `current` will behave just as it did before, marking the current location of the traversal. The new reference, which we will call `previous`, will always travel one node behind `current`. That way, when `current` stops at the node to be removed, `previous` will refer to the proper place in the linked list for the modification!

```
void remove(T item) {
    Node<T> *current = head;
    Node<T> *previous = NULL;
    bool found = false;
    // Step 1: traverse to find the item
    while (!found && current != NULL) {
        if (current->getData() == item) {
            found = true;
        } else {
            previous = current;
            current = current->getNext();
        }
    }
    // Step 2: unlink and FREE the node (C++ has no garbage collector!)
    if (found) {
        if (previous == NULL) {
            head = current->getNext();
        } else {
            previous->setNext(current->getNext());
        }
        delete current;
    }
}
```

Note the extra responsibility compared to Python: after unlinking, we must `delete` the node ourselves, or the memory leaks.

```
void remove(T item) {
    Node<T> *current = head;
    Node<T> *previous = NULL;
    bool found = false;
    // Step 1: traverse to find the item
    while (!found && current != NULL) {
        if (current->getData() == item) {
            found = true;
        } else {
            previous = current;
            current = current->getNext();
        }
    }
    // Step 2: unlink and FREE the node (C++ has no garbage collector!)
    if (found) {
        if (previous == NULL) {
            head = current->getNext();
        } else {
            previous->setNext(current->getNext());
        }
        delete current;
    }
}
```

Note the extra responsibility compared to Python: after unlinking, we must `delete` the node ourselves, or the memory leaks.

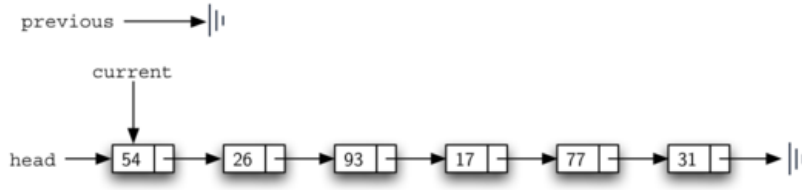
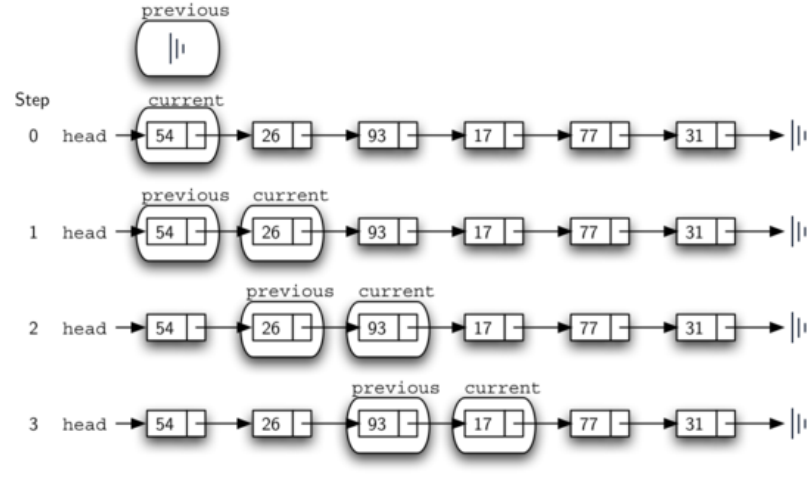
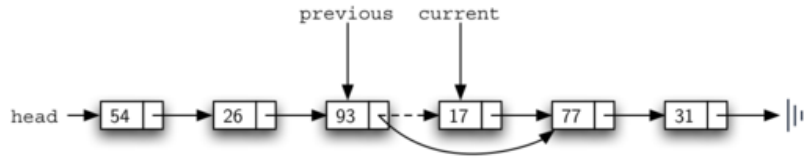
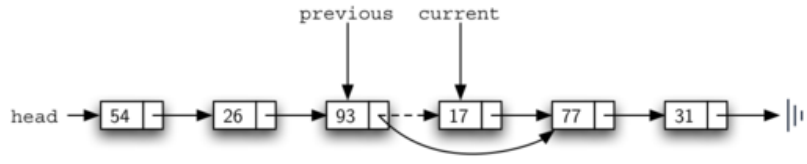

previous
current
head
54
26
93
17
77
31

Figure below shows the movement of `previous` and `current` as they progress down the list looking for the node containing the value 17.

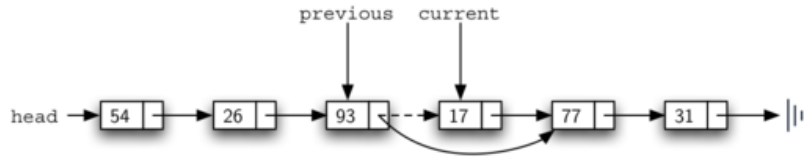
Figure below shows the movement of `previous` and `current` as they progress down the list looking for the node containing the value 17.




previous
current
head
54
26
93
17
77
31

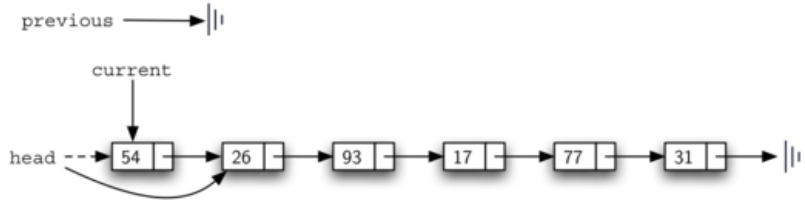


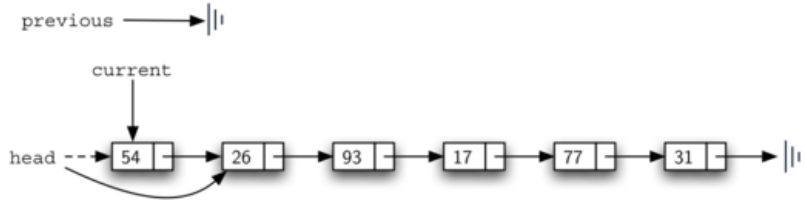
Once the searching step of the remove has been completed, we need to remove the node from the linked list. Figure above shows the link that must be modified.



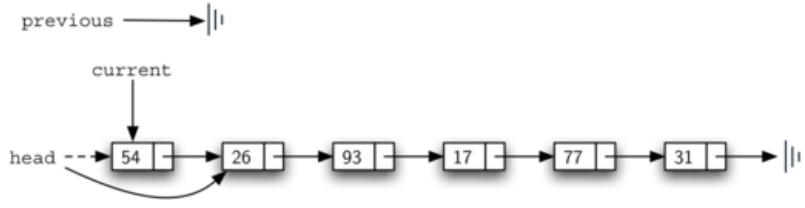
Once the searching step of the remove has been completed, we need to remove the node from the linked list. Figure above shows the link that must be modified.

However, there is a special case that needs to be addressed. If the item to be removed happens to be the first item in the list, then `current` will reference the first node in the linked list. This also means that `previous` will be `NULL`. We said earlier that previous would be referring to the node whose next reference needs to be modified in order to complete the removal. In this case, it is not previous but rather the head of the list that needs to be changed.


previous
current
head
54
26
93
17
77
31

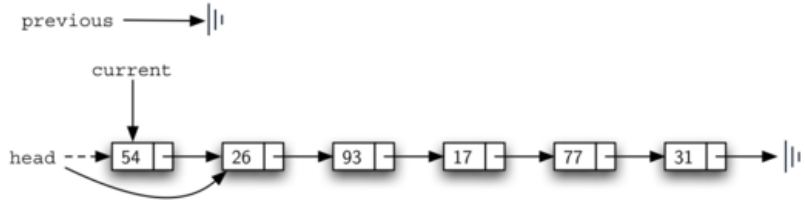


Line 13 allows us to check the special case described above. If `previous` didn't move, it will still have the value `NULL` when the loop breaks. In that case, the head of the list is modified to refer to the node after the current node (line 14), in effect removing the first node from the linked list. However, if previous is not `NULL`, the node to be removed is somewhere down the linked list structure.



Line 13 allows us to check the special case described above. If `previous` didn't move, it will still have the value `NULL` when the loop breaks. In that case, the head of the list is modified to refer to the node after the current node (line 14), in effect removing the first node from the linked list. However, if previous is not `NULL`, the node to be removed is somewhere down the linked list structure.

In this case the previous reference is providing us with the node whose next reference must be changed. Line 16 modifies the `next` property of the previous to accomplish the removal. Note that in both cases the destination of the reference change is `current.get_next()`.



Line 13 allows us to check the special case described above. If `previous` didn't move, it will still have the value `NULL` when the loop breaks. In that case, the head of the list is modified to refer to the node after the current node (line 14), in effect removing the first node from the linked list. However, if previous is not `NULL`, the node to be removed is somewhere down the linked list structure.

In this case the previous reference is providing us with the node whose next reference must be changed. Line 16 modifies the `next` property of the previous to accomplish the removal. Note that in both cases the destination of the reference change is `current.get_next()`.

You could also add a header node to simplify the process and this is left as exercise.

The complete templates are collected in `pythonds3/cppds/basic/linked_list.cpp`. Here is the whole `UnorderedList` in action:

In [2]:

```cpp
#include <iostream>
#include "dscpp/linked_list.hpp"
using namespace std;

int main() {
    UnorderedList<int> myList;
    myList.add(31); myList.add(77); myList.add(17);
    myList.add(93); myList.add(26); myList.add(54);

    cout << myList << endl;
    cout << myList.size() << endl;
    cout << boolalpha << myList.search(93) << endl;

    myList.remove(54);
    myList.remove(93);
    myList.remove(31);
    cout << myList << endl;
    return 0;
}
```

54 26 93 17 77 31

6

true

26 17 77

Notice `remove(54)`, `remove(93)`, `remove(31)` shrink the size step by step, and searching for a removed value now returns `false` — with each removed node's memory properly `delete`d.

In [ ]:
```
display_quiz(path+"unordered.json", max_width=800)
```

In the implementation of the add(item) method for an unordered linked list, what is the consequence of reversing the order of the linking steps (specifically, setting head = temp before calling temp->setNext(head))?

- ○ The code will not compile.
- ○ The original nodes in the list will be lost and can no longer be accessed (a memory leak in C++).
- ○ The list will automatically become a circularly linked list.
- ○ The new item will be added to the end of the list instead of the head.

4.5. The Ordered Linked List

We will now consider a type of list known as an ordered list. For example, if the list of integers shown in previous section were an ordered list (ascending order), then it could be written as 17, 26, 31, 54, 77, and 93.

We will now consider a type of list known as an ordered list. For example, if the list of integers shown in previous section were an ordered list (ascending order), then it could be written as 17, 26, 31, 54, 77, and 93.

The structure of an ordered list is a collection of items where each item holds a relative position that is based upon some underlying characteristic of the item. The ordering is typically either ascending or descending and **we assume that list items have a meaningful comparison operation that is already defined.**

Many of the ordered list operations are the same as those of the unordered list:

Many of the ordered list operations are the same as those of the unordered list:

- `OrderedList()` creates a new ordered list that is empty. It needs no parameters and returns an empty list.
- `add(item)` adds a new item to the list making sure that the order is preserved. It needs the item and returns nothing. Assume the item is not already in the list.
- `remove(item)` removes the item from the list. It needs the item and modifies the list. It will raise an error if the item is not present in the list.
- `search(item)` searches for the item in the list. It needs the item and returns a Boolean value.

- `is_empty()` tests to see whether the list is empty. It needs no parameters and returns a Boolean value.
- `size()` returns the number of items in the list. It needs no parameters and returns an integer.
- `index(item)` returns the position of an item in the list. It needs the item and returns the index. Assume the item is in the list.
- `pop()` removes and returns the last item in the list. It needs nothing and returns an item. Assume the list has at least one item.
- `pop(pos)` removes and returns the item at position pos. It needs the position and returns the item. Assume the item is in the list.

4.4 Implementing an Ordered Linked List

The ordered list of integers given above (17, 26, 31, 54, 77, and 93) can be represented by a linked structure as shown below.

The ordered list of integers given above (17, 26, 31, 54, 77, and 93) can be represented by a linked structure as shown below.

The ordered list of integers given above (17, 26, 31, 54, 77, and 93) can be represented by a linked structure as shown below.

Again, the node and link structure is ideal for representing the relative positioning of the items.

To implement the `OrderedList` class, we will use the same technique as seen previously with unordered lists. Once again, an empty list will be denoted by a head pointer to `NULL`:

To implement the `OrderedList` class, we will use the same technique as seen previously with unordered lists. Once again, an empty list will be denoted by a head pointer to `NULL`:

```
template <typename T>
class OrderedList {
    private:
        Node<T> *head;

    public:
        OrderedList() {
            head = NULL;
        }
};
```

To implement the `OrderedList` class, we will use the same technique as seen previously with unordered lists. Once again, an empty list will be denoted by a head pointer to `NULL`:

```
template <typename T>
class OrderedList {
    private:
        Node<T> *head;

    public:
        OrderedList() {
            head = NULL;
        }
};
```

As we consider the operations for the ordered list, we should note that the `is_empty()` and `size()` methods can be implemented the same as with unordered lists since they deal only with the number of nodes in the list without regard to the actual item values.

The methods, `search()` and `add()`, will require some modification.

The `search()` of an unordered linked list required that we traverse the nodes one at a time until we either find the item we are looking for or run out of nodes (`NULL`).

The `search()` of an unordered linked list required that we traverse the nodes one at a time until we either find the item we are looking for or run out of nodes (`NULL`).

It turns out that the same approach would work with the ordered list and no changes are necessary if the item is in the list. However, in the case where the item is not in the list, **we can take advantage of the ordering to stop the search as soon as possible.**

The `search()` of an unordered linked list required that we traverse the nodes one at a time until we either find the item we are looking for or run out of nodes (`NULL`).

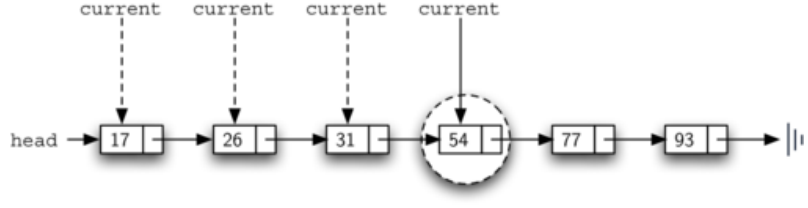
It turns out that the same approach would work with the ordered list and no changes are necessary if the item is in the list. However, in the case where the item is not in the list, **we can take advantage of the ordering to stop the search as soon as possible.**

For example, Figure below shows the ordered linked list as a search is looking for the value 45:

The `search()` of an unordered linked list required that we traverse the nodes one at a time until we either find the item we are looking for or run out of nodes (`NULL`).

It turns out that the same approach would work with the ordered list and no changes are necessary if the item is in the list. However, in the case where the item is not in the list, **we can take advantage of the ordering to stop the search as soon as possible.**

For example, Figure below shows the ordered linked list as a search is looking for the value 45:



The code below shows the complete `search()` method. It is easy to incorporate the new condition discussed above by adding another check (line 6):

The code below shows the complete `search()` method. It is easy to incorporate the new condition discussed above by adding another check (line 6):

```
bool search(T item) const {
    Node<T> *current = head;
    while (current != NULL) {
        if (current->getData() == item) {
            return true;
        } else if (current->getData() > item) {
            return false;   // passed the spot: stop early!
        }
        current = current->getNext();
    }
    return false;
}
```

The code below shows the complete `search()` method. It is easy to incorporate the new condition discussed above by adding another check (line 6):

```
bool search(T item) const {
    Node<T> *current = head;
    while (current != NULL) {
        if (current->getData() == item) {
            return true;
        } else if (current->getData() > item) {
            return false;   // passed the spot: stop early!
        }
        current = current->getNext();
    }
    return false;
}
```

We can continue to look forward in the list (line 3). If any node is ever discovered that contains data greater than the item we are looking for, we will immediately return `False`.

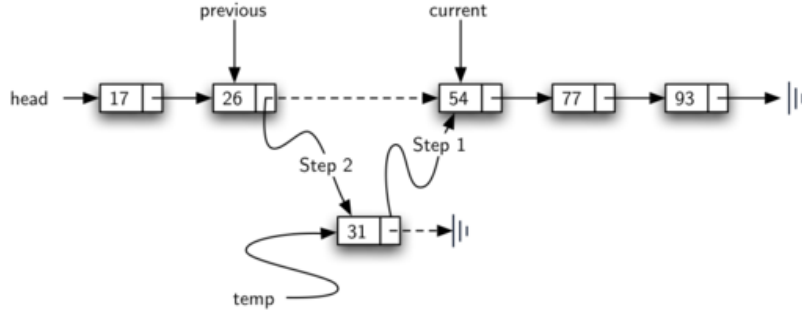
The most significant method modification will take place in `add()`. Recall that for unordered lists, **the add method could simply place a new node at the head of the list.** It was the easiest point of access.

The most significant method modification will take place in `add()`. Recall that for unordered lists, **the add method could simply place a new node at the head of the list.** It was the easiest point of access.

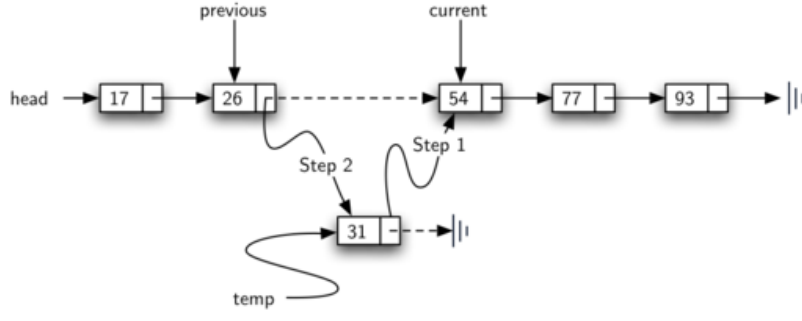
Unfortunately, this will no longer work with ordered lists. It is now necessary that we discover the specific place where a new item belongs in the existing ordered list.

Assume we have the ordered list consisting of 17, 26, 54, 77, and 93 and we want to add the value 31. The add method must decide that the new item belongs between 26 and 54:

Assume we have the ordered list consisting of 17, 26, 54, 77, and 93 and we want to add the value 31. The add method must decide that the new item belongs between 26 and 54:



Assume we have the ordered list consisting of 17, 26, 54, 77, and 93 and we want to add the value 31. The add method must decide that the new item belongs between 26 and 54:



As we explained earlier, we need to traverse the linked list looking for the place where the new node will be added!

As we saw with unordered lists, it is necessary to have an additional reference, again called `previous`, since `current` will not provide access to the node that must be modified.

As we saw with unordered lists, it is necessary to have an additional reference, again called `previous`, since `current` will not provide access to the node that must be modified.

```
void add(T item) {
    Node<T> *newNode = new Node<T>(item);
    if (head == NULL || head->getData() >= item) {
        newNode->setNext(head);
        head = newNode;
    } else {
        Node<T> *current = head;
        while (current->getNext() != NULL && current->getNext()->getData() <
item) {
            current = current->getNext();
        }
        newNode->setNext(current->getNext());
        current->setNext(newNode);
    }
}
```

The `OrderedList` class with methods discussed thus far can be found below:

The `OrderedList` class with methods discussed thus far can be found below:

The complete `OrderedList` is also in `pythonds3/cppds/basic/linked_list.cpp`:

The `OrderedList` class with methods discussed thus far can be found below:

The complete `OrderedList` is also in `pythonds3/cppds/basic/linked_list.cpp`:

In [4]:
```cpp
#include <iostream>
#include "dscpp/linked_list.hpp"
using namespace std;

int main() {
    OrderedList<int> myList;
    myList.add(31);
    myList.add(77);
    myList.add(17);
    myList.add(93);
    myList.add(26);
    myList.add(54);

    cout << myList << endl;
    cout << myList.size() << endl;
    cout << boolalpha << myList.search(93) << endl;
    cout << myList.search(100) << endl;
    return 0;
}
```

17 26 31 54 77 93

6

true

false

In [ ]:
```
display_quiz(path+"ordered.json", max_width=800)
```

What is the primary optimization in the search(item) method of an OrderedList compared to an UnorderedList when the item is not present in the list?

- ○ It stores an index for every node to allow for direct access.
- ○ It can jump backward to the head to re-scan the list more quickly.
- ○ It uses a binary search algorithm to find the item in logarithmic time.
- ○ It can stop the search early if it encounters a node containing a value greater than the target item.

4.4.1 Analysis of Linked Lists

To analyze the complexity of the linked list operations, we need to consider whether they require traversal.

Consider a linked list that has nodes. The `is_empty()` method is since it requires one step to check the head pointer for `NULL`. `size()`, on the other hand, will always require steps since there is no way to know how many nodes are in the linked list without traversing from head to end. Therefore, `size()` is .

To analyze the complexity of the linked list operations, we need to consider whether they require traversal.

Consider a linked list that has nodes. The `is_empty()` method is since it requires one step to check the head pointer for `NULL`. `size()`, on the other hand, will always require steps since there is no way to know how many nodes are in the linked list without traversing from head to end. Therefore, `size()` is .

Adding an item to an unordered list will always be since we simply place the new node at the head of the linked list. However, `search()` and `remove()`, as well as `add()` for an ordered list, all require the traversal process. Although on average they may need to traverse only half of the nodes, these methods are all since in the worst case each will process every node in the list.

In [ ]:
```
display_quiz(path+"complexity.json", max_width=800)
```

Which of the following UnorderedList methods has an average and worst-case time complexity of $O(1)$?

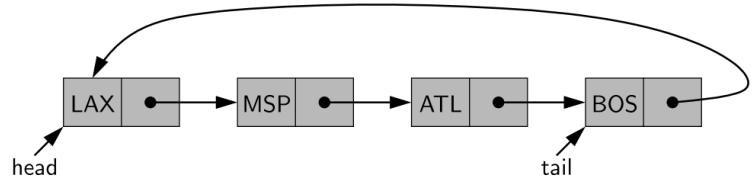
- ○ size()
- ○ remove(item)
- ○ search(item)
- ○ isEmpty()

A.1 Other types of Linked list

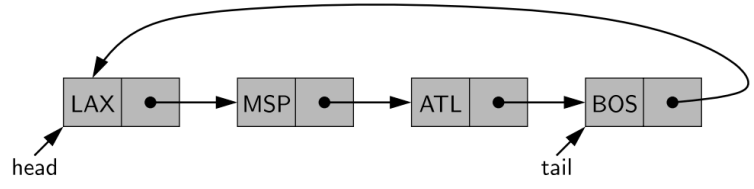
Circularly linked list

In the case of linked lists, there is a more tangible notion of a circularly linked list, as we can have the tail of the list use its next reference to point back to the head of the list, as shown below. We call such a structure a circularly linked list.

In the case of linked lists, there is a more tangible notion of a circularly linked list, as we can have the tail of the list use its next reference to point back to the head of the list, as shown below. We call such a structure a circularly linked list.



In the case of linked lists, there is a more tangible notion of a circularly linked list, as we can have the tail of the list use its next reference to point back to the head of the list, as shown below. We call such a structure a circularly linked list.



A circularly linked list provides a more general model than a standard linked list for data sets that are cyclic, that is, which do not have any particular notion of a beginning and end.

A circular view could be used, for example, to describe the order of train stops, or the order in which players take turns during a game. Even though a circularly linked list has no beginning or end, per se, we must maintain a reference to a particular node in order to make use of the list.

A circular view could be used, for example, to describe the order of train stops, or the order in which players take turns during a game. Even though a circularly linked list has no beginning or end, per se, we must maintain a reference to a particular node in order to make use of the list.

We use the identifier `current` to describe such a designated node. By setting `current = current.next`, we can effectively advance through the nodes of the list.

A circular view could be used, for example, to describe the order of train stops, or the order in which players take turns during a game. Even though a circularly linked list has no beginning or end, per se, we must maintain a reference to a particular node in order to make use of the list.

We use the identifier `current` to describe such a designated node. By setting `current = current.next`, we can effectively advance through the nodes of the list.

Ideal for efficient resource allocation and management or operation like `append()`!

Doubly linked list

In a singly linked list, each node maintains a reference to the node that is immediately after it. We have demonstrated the usefulness of such a representation when managing a sequence of elements. However, there are limitations that stem from the asymmetry of a singly linked list.

In a singly linked list, each node maintains a reference to the node that is immediately after it. We have demonstrated the usefulness of such a representation when managing a sequence of elements. However, there are limitations that stem from the asymmetry of a singly linked list.

We emphasized that we can efficiently insert a node at either end of a singly linked list, and can delete a node at the head of a list, **but we are unable to efficiently delete a node at the tail of the list.**

In a singly linked list, each node maintains a reference to the node that is immediately after it. We have demonstrated the usefulness of such a representation when managing a sequence of elements. However, there are limitations that stem from the asymmetry of a singly linked list.

We emphasized that we can efficiently insert a node at either end of a singly linked list, and can delete a node at the head of a list, **but we are unable to efficiently delete a node at the tail of the list.**

More generally, we cannot efficiently delete an arbitrary node from an interior position of the list if only given a reference to that node, because we cannot determine the node that immediately precedes the node to be deleted!

To provide greater symmetry, we define a linked list in which each node keeps an explicit reference to the node before it and a reference to the node after it. Such a structure is known as a doubly linked list.

To provide greater symmetry, we define a linked list in which each node keeps an explicit reference to the node before it and a reference to the node after it. Such a structure is known as a doubly linked list.

These lists allow a greater variety of -time update operations, including insertions and deletions at arbitrary positions within the list. We continue to use the term `next` for the reference to the node that follows another, and we introduce the term `prev` for the reference to the node that precedes it.

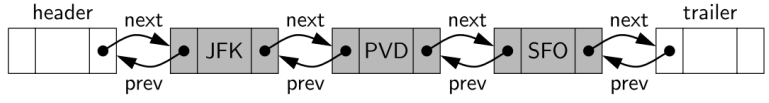
To provide greater symmetry, we define a linked list in which each node keeps an explicit reference to the node before it and a reference to the node after it. Such a structure is known as a doubly linked list.

These lists allow a greater variety of -time update operations, including insertions and deletions at arbitrary positions within the list. We continue to use the term `next` for the reference to the node that follows another, and we introduce the term `prev` for the reference to the node that precedes it.

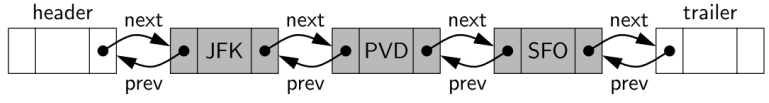
Ideal for algorithms that require backward traversal (e.g. Palindrome Check)!

A doubly linked list is shown

A doubly linked list is shown

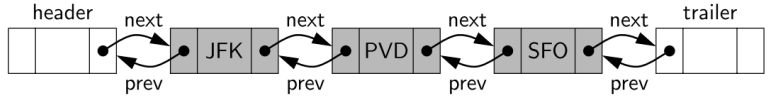


A doubly linked list is shown



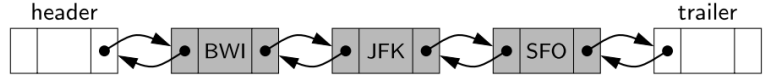
For a nonempty list, the header's next will refer to a node containing the first real element of a sequence, just as the trailer's prev references the node containing the last element of a sequence.

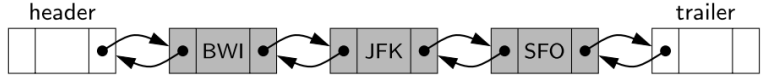
A doubly linked list is shown

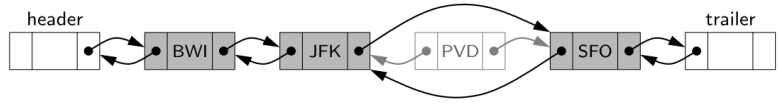


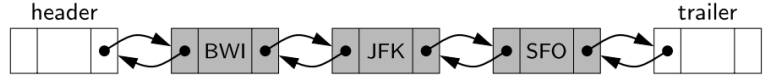
For a nonempty list, the header's next will refer to a node containing the first real element of a sequence, just as the trailer's prev references the node containing the last element of a sequence.

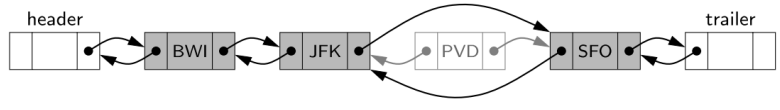
We can treat all insertions in a unified manner, because a new node will always be placed between a pair of existing nodes. In similar fashion, every element that is to be deleted is guaranteed to be stored in a node that has neighbors on each side.

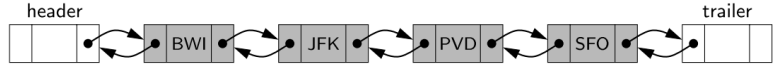

header
trailer
BWI
JFK
SFO

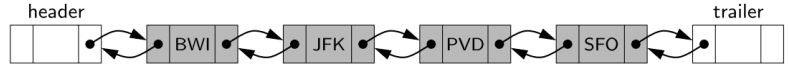

header
trailer
BWI
JFK
SFO

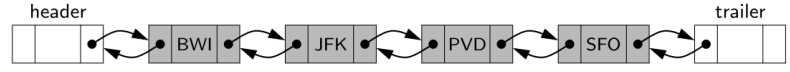

header
trailer
BWI
JFK
PVD
SFO

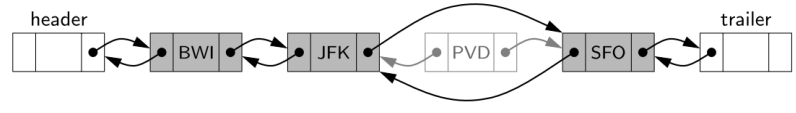

header
trailer
BWI
JFK
SFO

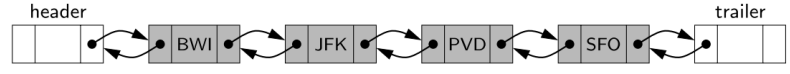

header
trailer
BWI
JFK
PVD
SFO

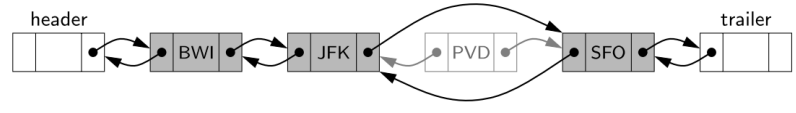

header
trailer
BWI
JFK
PVD
SFO

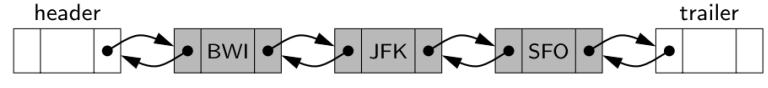

header
trailer
BWI
JFK
PVD
SFO


header
trailer
BWI
JFK
PVD
SFO

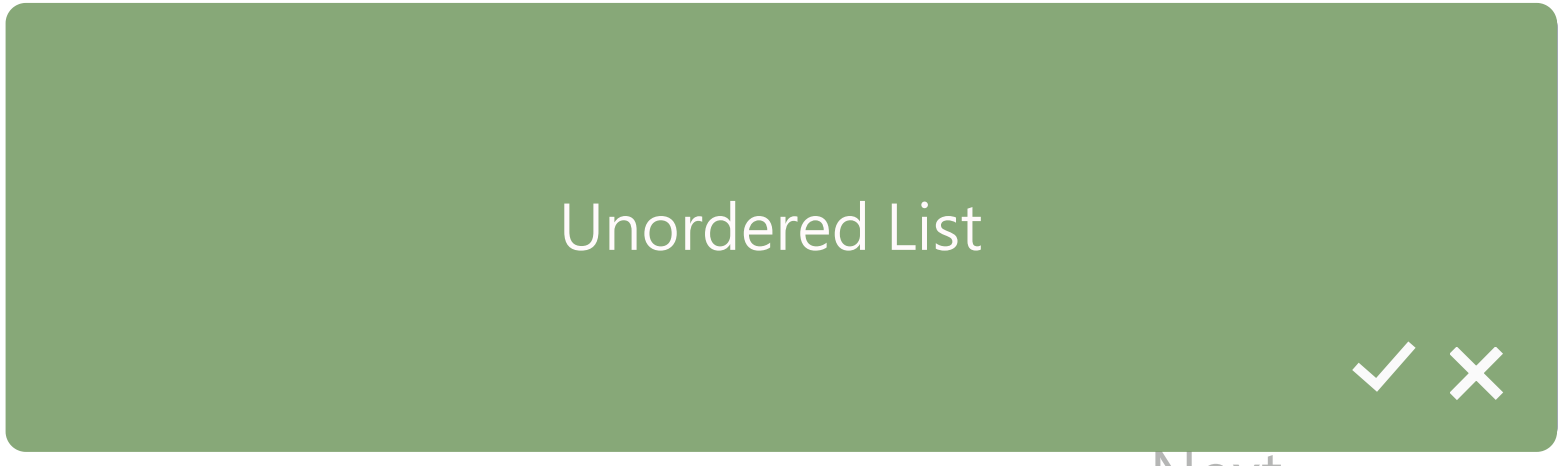

header
trailer
BWI
JFK
PVD
SFO


header
trailer
BWI
JFK
PVD
SFO


header
trailer
BWI
JFK
PVD
SFO


header
trailer
BWI
JFK
SFO

In [ ]:
```
from jupytercards import display_flashcards
fpath = "flashcards/"
display_flashcards(fpath + 'ch4.json')
```



References

1. Textbook: *Problem Solving with Algorithms and Data Structures using C++* (cppds), Chapter 4 (Linked Lists) — https://runestone.academy/ns/books/published/cppds/index.html